

Wolkite University
Department of Software Engineering

Introduction to Distributed System

Chapter 2: Architectures

April 18, 2025

Introduction

- Distributed systems are complex pieces of software of which the components are by definition dispersed across multiple machines.
- In order to manage their intrinsic complexity, distributed systems should be properly organized.
- Organization of a distributed system is mostly expressed in terms of its software components.
- Different ways to look at organization of distributed systems two obvious ones:
 - **Software architecture** logical organization (of software components and interconnections)
 - **System architecture** physical realization (the instantiation of software components on real machines)

Architectures

Software architecture - logical organization of distributed systems into software components

A architectural style is formulated in terms of:

- Types of **components**
 - The way that components are **connected** to each other
 - The data **exchanged** between components
 - How these elements are jointly **configured** into a system.
-
- **Software component** - a modular unit with well-defined, required and provided interfaces that is replaceable within its environment
 - **Connector** - a mechanism that mediates communication, coordination, or cooperation among component
 - e.g. a connector can be formed by the facilities for (remote) procedure calls, message passing, or streaming data.

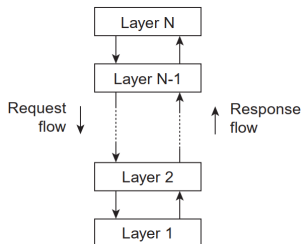
Architectural styles

Using components and connectors, we can come to various configurations, which in turn have been classified into four main architectural styles:

- Layered architectures
- Object-based architectures
- Data-centered architectures
- Event-based architectures

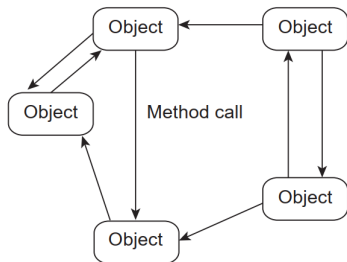
Architectural styles: Layered architectures

- Components are organized in a layered fashion where components of a layer only call components of the layer below, and are only called by the components of the layer above.
- "Mult-level client-server"
- Each layer acts as a
 - **Server:** Service provider to layer "above"
 - **Client:** service consumer of layers(s) "below":
- Connectors are protocols of layer interaction



Architectural styles: Object-based architectures

- Each object corresponds to a component and these components are connected through a remote procedure call mechanism (client-server paradigm)



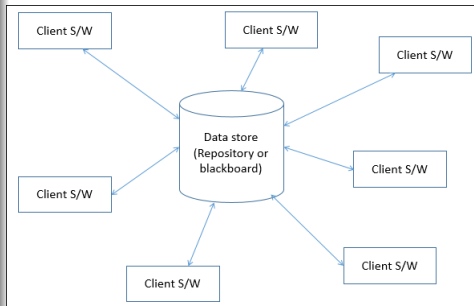
Note

- Layered style used for client-server system
- Object-based style for distributed object system
- Layered and object-based style are the most important style for distributed system today

Architectural styles: Data-centered architectures

Data-centered architectures

- Access and update of data stored is the main purpose of the system.
- Processes communicate info primarily by reading and modifying data in some shared repository;
 - e.g., a shared distributed file system, database
- Components:
 - Data store
 - Client interact with store
- Connectors:
 - Queries



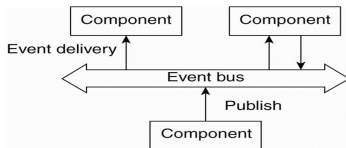
Architectural styles: Data-centered architectures

- Clients are quite independent of each other
 - client can be modified without affecting others
 - new client can be easily added
- Issues:
 - How information is represented
 - How events are handled,
 - how the shared repository behave in response to interaction,
 - how process interact with/ through the shared repository.
- As important as the layered and object-based architectures
- Examples:
 - Web-based systems, are largely data-centric
 - Many networked application have been developed that relay on a shared distributed file system in which virtually all communication through files.

Architectural styles: Event-based architectures

Event-based architectures

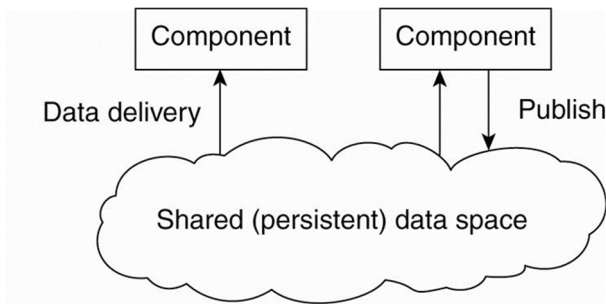
- Processes communicate through an event bus through which events are propagated and possibly carrying data along.



- Components:
 - Can be an instance of class or simply a module
 - Publisher:** individual components announce data that they wish to share with their subscribers.
 - Subscriber:** individual components register their interest for published data.
- Connectors:
 - Event buses

Architectural styles: shared data spaces

- Event-based architectures can be combined with data-centered architectures, yielding what is also known as **shared data spaces**.
- processes are decoupled in time

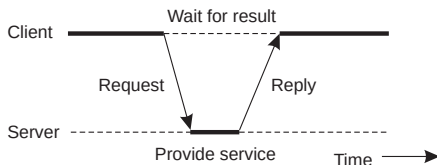


System Architectures

- Deciding on software components, their interaction, and their placement leads to an instance of a software architecture, also called a **system architecture**.
- System architectures can usually be characterized as examples of one or more architectural styles
- The realization of an architecture may be:
 - **Centralized** - most components located on a single machine
 - **Decentralized** - most machines have approximately the same functionality
 - **Hybrid** - some combination

Centralized Architectures

- Logical separation of functions into **client** (requesting process) and **server** (responder)
- Hierarchical (or vertical) organization
- Traditional client-server structure
 - **Client**: uses the service by sending a request and waiting for the reply
 - **Server**: a process implementing a certain service. E.g., a database service



General interaction between a client and a server

Centralized Architectures

Communication between client and server can be:

- By a **connectionless protocol** if the underlying network is fairly reliable; efficient since there is no much overhead
 - but assuring reliability is difficult
 - when messages are lost or corrupted let the client send the request again; applicable only for idempotent operations
 - an operation is **idempotent (unchanged)** if it can be repeated multiple times without harm; e.g., reading a record in a database
 - **We will see detail in Fault tolerant**
- By **reliable connection-oriented protocol** if the underlying network is unreliable
 - establishing and terminating connections is expensive

Centralized Architectures: Layered architecture for Client-Server systems

Traditional three-layered view

- **User-interface layer** contains units for an application's user interface
- **Processing layer** contains the functions of an application, i.e. without specific data
- **Data layer** contains the data that a client wants to manipulate through the application components
 - Data usually is persistent; exists even if no client is accessing it.
 - file or database system

Observation

This layering is found in many distributed information systems, using traditional database technology and accompanying applications.

Centralized Architectures: Layered architecture for Client-Server systems

Traditional three-layered view

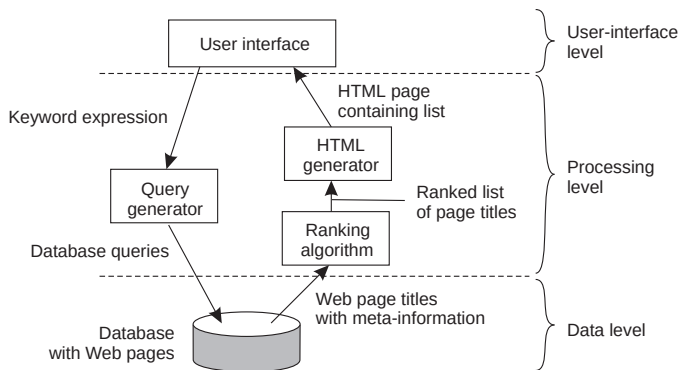
- **User-interface layer** contains units for an application's user interface
- **Processing layer** contains the functions of an application, i.e. without specific data
- **Data layer** contains the data that a client wants to manipulate through the application components
 - Data usually is persistent; exists even if no client is accessing it.
 - file or database system

Observation

This layering is found in many distributed information systems, using traditional database technology and accompanying applications.

Centralized Architectures: Layered Architecture for client-server systems

e.g., the general organization of an Internet search engine into three different layers



- How to physically distribute a client-server application across several machines $\Rightarrow$ **Multitiered Architectures**

Centralized Architectures: Multi-Tiered Architectures

Logical Architecture vs. Physical Architecture

- **Layer** and **tier** are roughly equivalent terms, but **Layer** typically implies software and **tier** is more likely to refer to hardware.
- Logical organization is not physical organization.
- Physical architecture may or may not match the logical architecture.
- Clients and servers could be placed on the same node, or be distributed according to several different typologies.
- How do we map the software architecture to system hardware?
 - Correspondence between logical software modules and actual computers
- Could have various architectures:
 - **Single-tiered**: dumb terminal/mainframe configuration
 - **Two-tiered**: client/single server configuration
 - **Three-tiered**: each layer on separate machine

Centralized Architectures: Multi-Tiered Architectures

Two-Tiered Architecture

- Where are the three application-layers placed? On the client machines, or on the server machines?
- A range of possible solutions:
 - **Thin-Client** - Server provides processing and data management; client provides simple graphical display
 - **Pros**: easier to manage, more reliable, client machines don't need to be so large and powerful
 - **Con**: perceived performance loss at client
 - **Fat-Client** - All application processing and some data resides at the client
 - **Pros**: reduces work load at server; more scalable
 - **Con**: harder to manage by system admin, less secure
 - Other solutions in between thin-client and fat-client

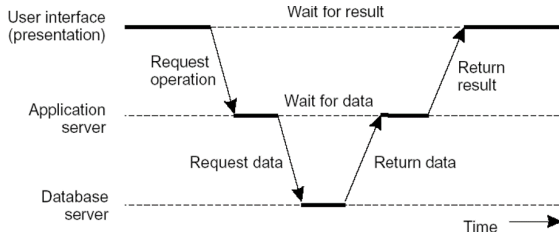
Traditional two-tiered configurations



Centralized Architectures: Multi-Tiered Architectures

Three tiered architecture

- Server-side solutions are becoming increasingly more distributed as a single server is being replaced by multiple servers running on different machines.
- A server may sometimes need to act a client.
- **Three-tiered**: each of the three layers corresponds to three separate machines. E.g., Web Servers, Distributed transaction processing



Decentralized Architectures: Vertical vs. Horizontal

Vertical Distribution

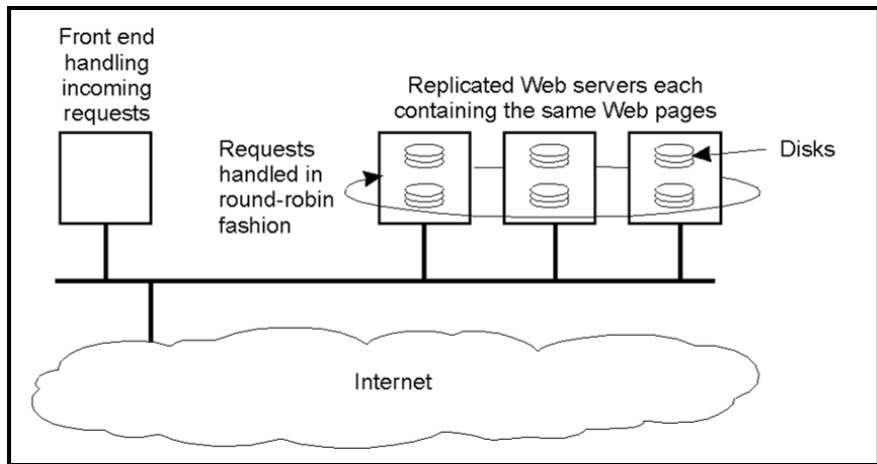
- Multi-tiered client-server architectures are a direct consequence of dividing applications into a user-interface, processing components, and a data level.
- Logical organization is mapped onto the tiers.
- In many cases, distributed processing amounts at building a client-server application according to a multi-tiered architecture.
- This is typically called **vertical distribution**.

Horizontal Distribution

- Sometimes, the physical distribution of the clients and the servers is what actually counts.
- A client or server may be physically split up into logically equivalent parts, but each part is operating on its own share of the complete data set, thus balancing the load.
- The architecture in this case is decentralized and forms **horizontal distribution**.
- **Peer-to-peer systems** are a class of modern architectures that support horizontal

Decentralized Architecture: Vertical vs. Horizontal

Horizontal Distribution



Decentralized Architectures: Peer-to-Peer Systems

- Peer-to-peer systems have better load balancing and more resistant to denial of-service attacks, but harder to manage than client-server systems.
- Nodes act as both client and server; interaction is symmetric.
- Their organization can be:
 - **Structured P2P**: nodes are organized following a specific distributed data structure
 - **Unstructured P2P**: nodes have randomly selected neighbors
 - **Hybrid P2P**: some nodes are appointed special functions in a well organized fashion
- **Overlay networks** connect nodes in the P2P system.
 - Define the structure between nodes in the system.
 - Allow nodes to route requests to locations that may not be known at time of request.

Hybrid Architectures

- Many distributed systems require properties from both client-server and peerto-peer architectures.
- So, they put together features from both centralized and decentralized architectures, resulting in [hybrid architectures](#).
- Some nodes are appointed special functions in a well-organized fashion
- Edge-server systems:
 - E.g., ISPs, which act as servers to their clients, but cooperate with other edge servers to host shared content
- Collaborative distributed systems:
 - E.g., BitTorrent, which supports parallel downloading and uploading of chunks of a file. First, interact with client-server system, and then operate in decentralized manner.

End of chapter 2

Any Question ?